

URL Shortener — Data Flow Analysis

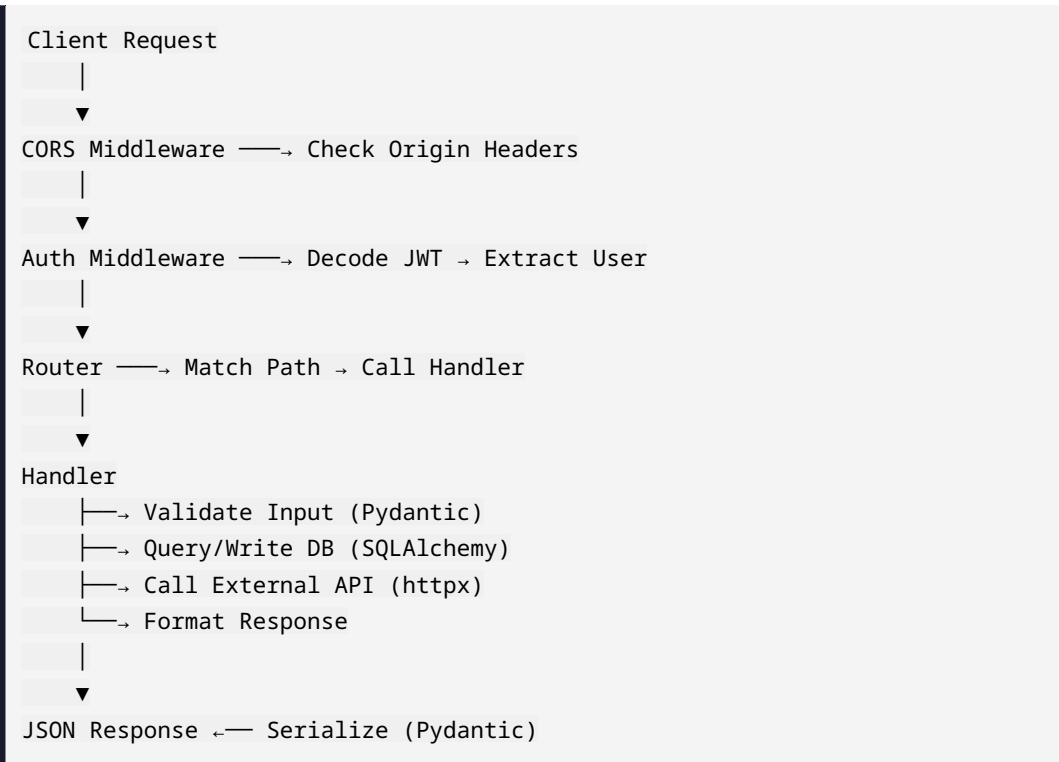
Project: URL Shortener
Technology: Python, FastAPI, SQLAlchemy, SQLite, Pydantic
Location: /home/dominic/Documents/fastapiurl
Report #: 15/20

Request Flow

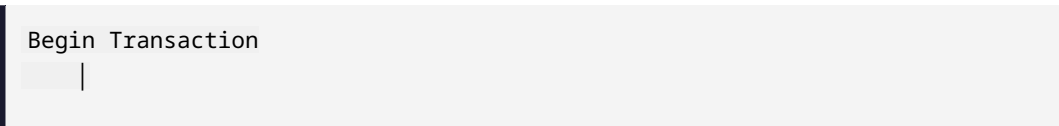
Client → POST /shorten → FastAPI → SQLite (insert)
Client → GET /{code} → FastAPI → SQLite (query) → 302 Redirect
Client → GET /{code}/stats → FastAPI → SQLite (count) → JSON

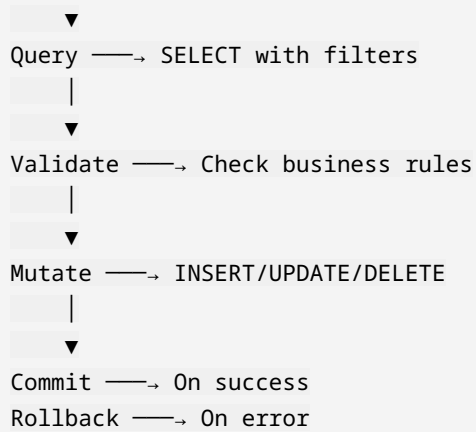
Data Flow Analysis

Request Data Flow



Database Transaction Flow





WebSocket Data Flow (Queue Module)



External API Data Flow (AI Chat Proxy)



Data Transformations

1. **URL shortening:** Long URL → short_code (base62 hash)
2. **Auth:** Password → bcrypt hash → DB
3. **RAG:** PDF → Text → Chunks → TF-IDF vectors
4. **Queue:** Task request → DB row → WebSocket progress
5. **Chat:** Message → JSON → API call → SSE stream